Project by

Shuvro Ahmed
Umme Humaira Himi
Ahmed Hassan

POLITECNICO MILANO 1863

Design Document

for

Best Bike Paths

Politecnico di Milano – A.Y. 2025/2026

Deliverable:	DD
Title:	Design document
Authors:	Shuvro Ahmed, Umme Humaira Himi, Ahmed Hassan
Version:	1.0
Date:	7-January-2026
Download page:	https://github.com/H-I-M-I/HassanAhmedHimi
Copyright:	Copyright © 2025, Shuvro Ahmed, Umme Humaira Himi, Ahmed Hassan – All rights reserved

Contents

Table of Contents **2**

List of Figures **3**

List of Tables **3**

1 Introduction **4**
1.1 Purpose 4
1.2 Scope 4
1.3 Definitions 4
1.4 Acronyms 4
1.5 Abbreviations 5
1.6 Revision History 5
1.7 Reference Documents 5
1.8 Document Structure 5

2 Architectural Design **6**
2.1 Overview: High level components and their interaction 6
2.2 Component view 7
2.3 Deployment view 9
2.4 Runtime view 11
2.5 Component interfaces 14
2.6 Selected architectural styles and patterns 16
2.7 Other design decisions 17

3 User Interface Design **20**
3.1 Overview 20
3.2 Main User Interfaces 20
3.3 Key Screens and Workflows 22
3.4 Interface Conventions and Legends 23
3.5 Accessibility and Interaction Notes 23
3.6 Navigation Flow 24

4 Requirements Traceability **25**

5 Implementation, Integration and Testing **28**
5.1 Implementation Strategy 28
5.2 Feature Identification 28
5.3 Component Testing 28
5.4 Incremental Integration Strategy 29
5.5 Thread-Based Integration 29
5.6 Final System Integration 30
5.7 Testing Summary 30

6 Effort Spent **31**
6.1 Shuvro Ahmed 31
6.2 Umme Humaira Himi 31
6.3 Ahmed Hassan 32

7 References **33**

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

List of Figures

1 Component interactions in the BBP architecture . . . 7
2 BBP component diagram (annotated) . . . 8
3 Deployment View of Best Bike Path . . . 10
4 Runtime View of UC5+UC6 . . . 11
5 Runtime View of UC1 . . . 12
6 Runtime View of UC3 . . . 13
7 Runtime View of UC4 . . . 18
8 Runtime View of G6/F6 . . . 19
9 Main map interface of the BBP mobile application . . . 20
10 Trip recording interface during a cycling activity . . . 21
11 Path search and visualization interface . . . 21
12 Interface for confirmation of automatically detected issues . . . 22
13 Component testing using a test driver and stubs. . . . 29
14 Incremental integration strategy with progressive replacement of stubs. . . . 29
15 Thread-based integration of the Path Search and Visualization feature. . . . 30
16 Final system configuration after full integration. . . . 30

List of Tables

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

1 Introduction

1.1 Purpose

The Best Bike Paths (BBP) system is intended to assist bicycle riders in determining and using the best available bike path given a city's context including factors such as path safety, user-friendly design, obstacles etc. Users can provide information about the bike path by logging trips or manually and automatically reporting obstacles faced during cycling. The objective of this Design Document is to show the structure of BBP system in order meet these goal, report about architectural organization and design decisions which have been made to ensure functionalities described in Requirement Analysis and Specification Document (RASD). The aim of this paper is to be a detailed specification for developers and testers in following phases as implementation, integration and testing phase.

1.2 Scope

The BBP system collects, administers and displays bike path and cycling information. More specifically, the system allows for the capturing of bike trips, its includes and validating path-related information, and generating optimal bike paths between any two points based on an aggregated user's input. This Design Document proposes how the software architecture should be, and what components are necessary to implement these functionalities, its interactions and their interfaces. This is intended to provide a clear reference for developers and testers during the subsequent phases of implementation, integration, and testing.

1.3 Definitions

This subsection reports the definitions of the main concepts used throughout this Design Document. Additional definitions introduced during the requirements analysis phase are described in the RASD.

- **Bike Path**: A path suitable for cycling, either because it includes a dedicated bike lane or because car traffic is limited and compatible with bicycle usage.
- **Trip**: A cycling activity recorded by a registered user, consisting of spatial and temporal data collected during movement.
- **Publishable Information**: Information about bike paths or obstacles that has been confirmed by the user and made visible to the community.
- **Component**: A self-contained software unit that implements a specific responsibility and interacts with other components through well-defined interfaces.
- **Thread**: A complete execution flow that spans multiple components to deliver a user-visible functionality.
- **Stub**: A simulated component used during testing to replace external dependencies of the component under test.
- **Test Driver**: A software element that invokes a component under test and controls its execution during testing.

1.4 Acronyms

- **BBP**: Best Bike Paths
- **DD**: Design Document

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

- **RASD**: Requirement Analysis and Specification Document
- **UI**: User Interface
- **API**: Application Programming Interface

1.5 Abbreviations

- **GPS**: Global Positioning System
- **DB**: Database

1.6 Revision History

Version 1.0 : 07/01/2026.

1.7 Reference Documents

This subsection lists the documents that have been consulted and referenced during the preparation of this Design Document and that provide the basis for the architectural and design choices described in the following sections.

- Requirement Analysis and Specification Document (RASD) - Best Bike Paths
- Software Engineering 2 - Assignment R&DD, A.Y. 2025–2026
- Software Engineering 2 course material and lecture slides

1.8 Document Structure

This document is structured as follows:

1. **Introduction**: provides a brief description of the project, focusing on the motivations and goals to be achieved through its development, as defined in the RASD.
2. **Architectural Design**: describes the architecture of the system at different levels of granularity, including the main components, their interactions, and the adopted architectural patterns.
3. **User Interface Design**: presents the organization of the user interface of the platform and describes how users interact with the system.
4. **Requirements Traceability**: explains how the requirements defined in the RASD are satisfied by the design elements introduced in this document.
5. **Implementation, Integration and Test Plan**: provides an overview of the implementation strategy, integration activities, and testing approach adopted for the system.
6. **Effort Spent**: reports the effort spent by each group member in the preparation of this document.
7. **Bibliography**: contains the references to the documents and tools used to produce this Design Document.

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

2 Architectural Design

2.1 Overview: High level components and their interaction

Purpose Best Bike Paths (BBP) is a client-server system that supports: trip recording and review, manual and automatic contribution of path issues, validation of auto-detected issues, path search with ranked alternatives, and merging/moderation of crowd-sourced path status. These capabilities are required by the assignment and are specified in the RASD scenarios and requirements.

System-level architecture (logical 3-tier) BBP follows a logical three-tier architecture (separation of responsibilities, not a deployment claim):

- Presentation tier (Clients): mobile application that offers UI/UX for cyclists/users to record their trips, choose best path, report issues, etc., with an optional Admin Panel web app for moderators.
- Business tier (Backend): a modular monolith backend exposing REST APIs over HTTPS and implementing all BBP business rules (trip/issue/merge/ranking) as modules.
- Data tier (Storage & caching): persistent storage for different entities (users, trips, ...); also a cache for read-heavy/expensive results.

BBP also relies on external providers (maps/routing, weather, traffic). These providers are outside the system boundary; the backend accesses them through dedicated adapter components (described below).

Architectural choice and rationale (modular backend + async support) BBP implements the business tier as a modular monolith: a single backend application whose internal modules align with major domain responsibilities (Trip, Issue/Validation, Aggregation/Moderation, Path Search/Ranking, Identity & Access). This is well-suited for a startup/new application because it keeps the domain logic cohesive and easy to evolve while still enforcing clear boundaries between responsibilities.

The backend is complemented by asynchronous processing for tasks that are slow, bursty, or dependent on external services (e.g., enrichment and automatic detection). Instead of blocking interactive requests, the backend can enqueue work and let an internal worker execute it, improving responsiveness and robustness without requiring a fully distributed microservice architecture.

This solution is preferred over a microservices-based distributed architecture because BBP contains cross-cutting domain invariants—most notably the aggregation workflow that maintains a single publishable status per segment and resolves conflicts through confirmations and freshness—that are simpler to enforce when core domain updates occur within a single application boundary. Microservices could still realize the same functionality, but would typically introduce additional coordination mechanisms (messaging, idempotent event handling, and reconciliation) to keep derived "publishable" state aligned with incoming reports/confirmations, increasing design and operational complexity for a startup/new application. At the same time, the modular decomposition keeps boundaries explicit and enables future evolution towards independently deployable services if specific modules (e.g., detection/enrichment) require it.

Backend internal architecture (layers inside the Business tier) Within the backend (tier 2), the codebase is internally organized into layers (Interface, Domain, Repository and Adapter).

Asynchronous processing Some backend activities can take significant time and are not required to complete before the user receives a response. To keep interactive operations responsive, BBP executes these activities asynchronously: the backend records the core data immediately and schedules the additional processing to be handled in the background by a worker. In BBP this is primarily used to (i) enrich uploaded trips with weather/traffic context by contacting external providers without delaying the upload,

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

and (ii) run automatic issue detection on the uploaded trip trace and later make the resulting candidate issues available to the user in the "items to confirm" workflow.

External services BBP relies on external providers that are outside the system boundary and are accessed by the backend through adapter components. These providers supply:

- Map/Geocoding/Routing functionality (map visualization, geocoding/reverse-geocoding, route alternatives),
- Weather information (trip enrichment and optional ranking context),
- Traffic information (trip/path context and optional ranking or presentation).

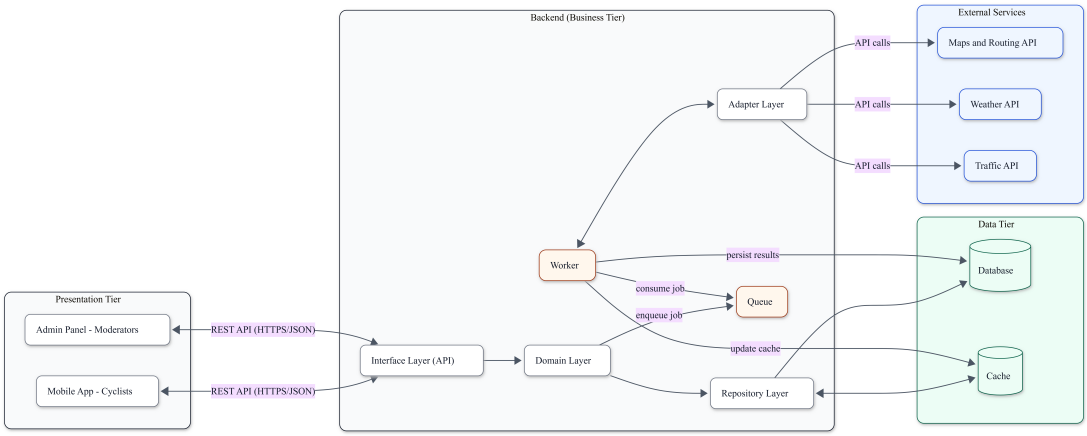


Figure 1: Component interactions in the BBP architecture

2.2 Component view

Component structure and system boundary This section describes the main components of BBP and how they collaborate to deliver the system features. The component view follows the same logical three-tier structure introduced above: (i) a presentation tier (clients), (ii) a business tier (backend components that implement the application logic), and (iii) a data tier (persistence and caching). It also highlights where BBP interacts with external services outside the system boundary.

Presentation tier (Clients)

- **Mobile App**: client application used by cyclists to record trips and navigate. It supports manual issue reporting and confirmation of automatically detected issues. The app stores a limited local state (e.g., trips and reports) to remain usable when connectivity is unavailable; once the network is restored, it synchronizes the pending data with the backend.
- **Admin Panel**: web interface reserved for moderators to handle unresolved conflicts and execute moderation actions.

Clients access BBP through a **public Client API** exposed by the backend.

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

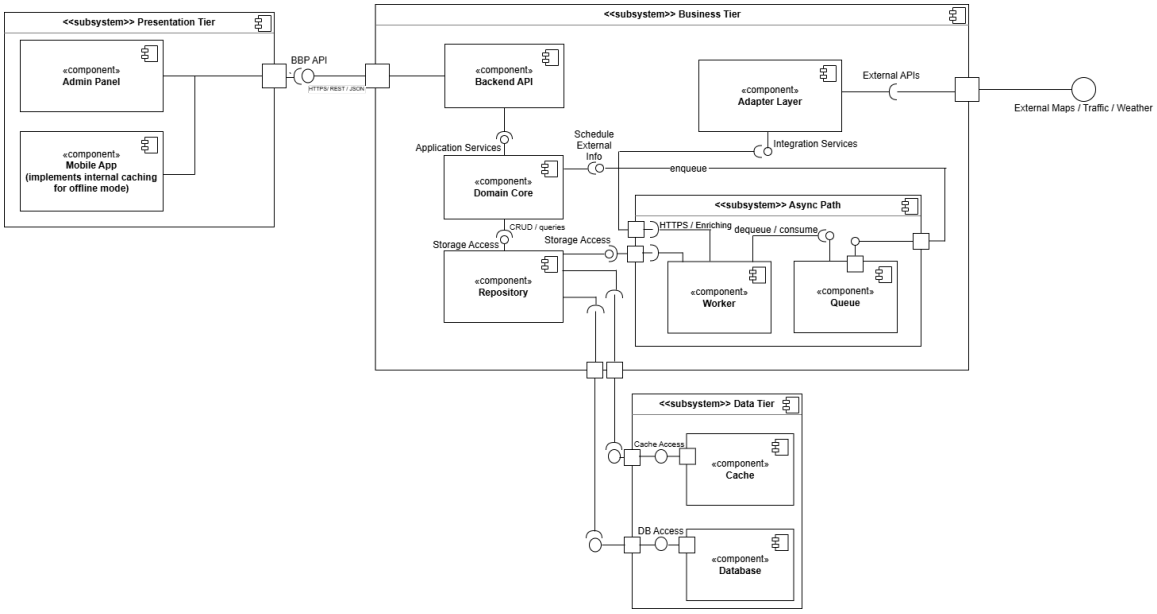


Figure 2: BBP component diagram (annotated)

Business tier (Backend)

- **Backend API**: exposes BBP REST endpoints over HTTPS using JSON. It validates incoming requests, enforces authentication and authorization, and routes valid requests to the core application logic.
- **Domain Core**: implements the application rules and manages the main workflows (e.g., trip management, issue lifecycle, and validation). It also performs aggregation and segment status updates, and supports path search features such as path ranking. Time-consuming operations that do not need to complete within the request/response cycle can be delegated to asynchronous processing.
- **Repository**: persistence boundary used by the Domain Core and background worker. It encapsulates access to the database and cache, keeping storage concerns separate from business logic.
- **Async Path (Queue + Worker)**:
  - **Queue**: stores background jobs emitted by the Domain Core (e.g., enrichment and detection tasks).
  - **Worker**: consumes jobs from the queue, executes the required processing, and persists results through the Repository.
- **Adapter Layer (Gateway)**: integration boundary for third-party providers. It hides provider-specific APIs and formats, handles failures and retries as needed, and returns normalized data to backend components that require external information.

Data tier (Storage & caching)

- **Database**: persistent storage for core entities such as users, trips, issues, confirmations, and the computed publishable segment status.
- **Cache**: used to reduce latency for read-heavy or expensive queries and to avoid repeated computation and repeated provider calls. The cache is accessed through the Repository; it may be refreshed or updated by background processes to keep frequently used data up to date.

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

External services (outside system boundary) BBP depends on external providers for:

- **Maps & Routing**: geocoding/reverse-geocoding, route alternatives, and other map-related data,
- **Weather**: trip enrichment and optional ranking context,
- **Traffic**: trip/path context and optional ranking or presentation.

These services are accessed only through the Adapter Layer.

Connectors and interactions

- **Client-to-backend**: the Mobile App and Admin Panel communicate with the Backend API via the public Client API using HTTPS (REST + JSON).
- **Synchronous backend flow**: the Backend API invokes the Domain Core for request handling; the Domain Core persists and retrieves state through the Repository.
- **Asynchronous backend flow**: the Domain Core enqueues background jobs to the Queue; the Worker consumes them, executes the processing, and stores outcomes via the Repository.
- **Third-party integrations**: external API calls are performed by the Adapter Layer, which contacts providers (Maps/Routing, Weather, Traffic) and normalizes their responses for backend use.

Constraints captured by the component view

- **No direct access to internal resources**: clients do not directly access the data tier or external services; all access is mediated by the backend (and, for third parties, through the Adapter Layer).
- **Persistence encapsulation**: persistence is primarily handled through the Repository (database + cache), keeping storage concerns separate from the Domain Core and background worker.
- **Third-party isolation**: all provider dependencies are isolated inside the Adapter Layer to limit coupling between the core system and external APIs.
- **Responsiveness**: time-consuming and non-interactive work is handled by the Queue/Worker path to keep request/response operations responsive.

2.3 Deployment view

For, deployment view diagram, we have used the Draw.io [2] software.

Smartphone (Mobile Device)
A smartphone used by registered bikers to access BBP through the mobile application. The device provides access to GPS and inertial sensors used during trip recording and automatic issue detection. In case of intermittent connectivity, the mobile app can temporarily store trip and sensor data locally and later synchronize it when the network becomes available.

Computer / Browser (Optional Web Access)
A computer (or any device) used by users to access BBP through a web browser. The browser interacts with the system through HTTPS requests to visualize paths on a map and browse public information.

Internet / Network Connection
The connection between clients and the backend is established over HTTPS/TLS. The same network boundary separates BBP's trusted backend infrastructure from external third-party services.

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

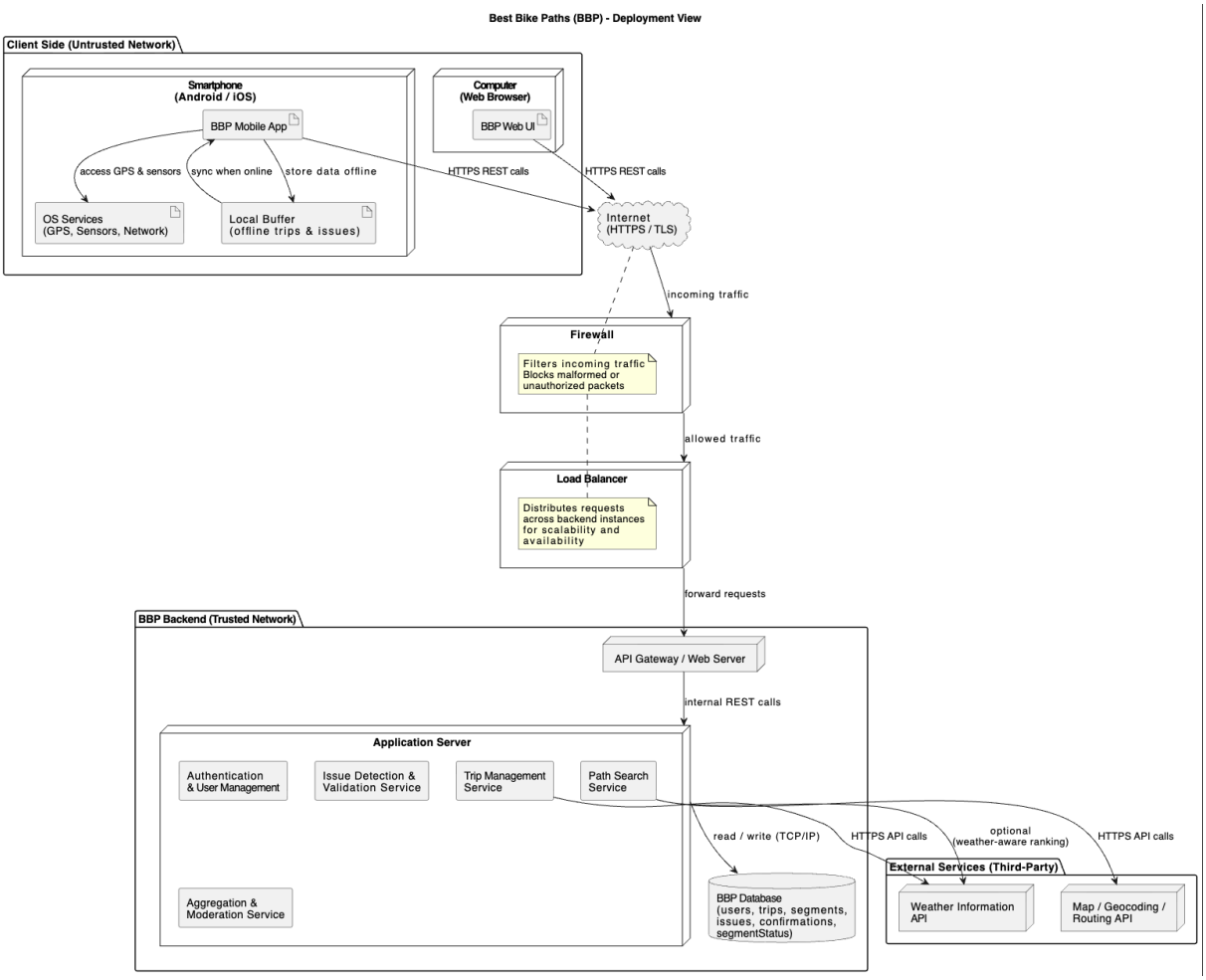


Figure 3: Deployment View of Best Bike Path

Firewall / Perimeter Security
A logical firewall protects the backend infrastructure by filtering incoming traffic and allowing only expected HTTPS communication toward BBP's exposed endpoints. This component represents the security boundary between the public Internet and the trusted backend network.

Load Balancer / API Entry Point
The load balancer is responsible for routing client requests to available instances in a scalable way. It avoids overloading a single server and provides a stable access point for both the mobile app and the web UI.

API Gateway / Web Server
The API gateway terminates HTTPS connections and exposes the REST-like API endpoints used by clients. It routes requests to the backend application services and can serve static web resources without involving application logic when applicable.

Application Server (Backend Services)
The application server hosts BBP's application logic. It manages authentication, trip recording and review, path search and ranking, issue reporting/validation, and the aggregation of contributions from multiple users. It is also the component responsible for interacting with the persistence layer and orchestrating calls to external services when required by the use cases.

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

Database Server
The database stores persistent information required by BBP, including users, trips and their statistics, recorded path segments, issues and confirmations, and the aggregated publishable status of segments. The database is only accessed by the application server, ensuring that clients never directly interact with stored data.

Map / Geocoding / Routing Service (External)
A third-party external service used by BBP to translate user locations into geographic coordinates (geocoding) and to compute candidate routes between origin and destination. BBP uses its responses to construct and visualize candidate bike paths and to support ranking.

Weather Information Service (External)
A third-party external service used by BBP to retrieve meteorological data (e.g., temperature, wind). BBP uses this data to enrich trip records and as an input factor when presenting and ranking paths.

2.4 Runtime view

In this section we describe how BBP components interact at runtime to accomplish key tasks. The following sequence diagrams are design-level. Lifelines represent architectural components (client, gateway, backend services, database, external services) rather than a single system black box. For readability, authentication checks and minor validation steps are omitted unless central to the interaction.

UC5/UC6 Search Path + View Suggested Paths

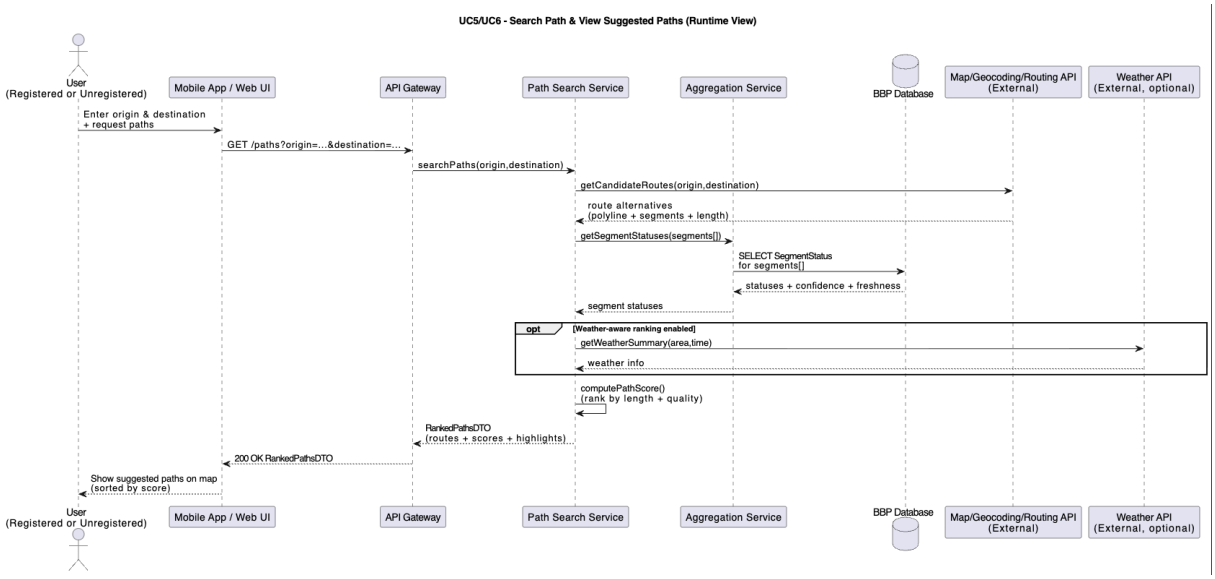


Figure 4: Runtime View of UC5+UC6

This sequence shows how BBP answers a path query between an origin and destination. The client sends a request to the backend through the API gateway. The Path Search Service retrieves candidate routes from an external routing provider and then requests the publishable, merged segment information (SegmentStatus) for the segments involved in those routes. The Aggregation Service fetches the relevant SegmentStatus data from the database and returns it to the Path Search Service, which computes a path score combining route effectiveness and reported quality. Optionally, weather data can be retrieved from

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

an external service and included as an additional ranking factor. Finally, the ranked paths are returned to the client for visualization on the map.

UC1 Record Trip + Weather Enrichment

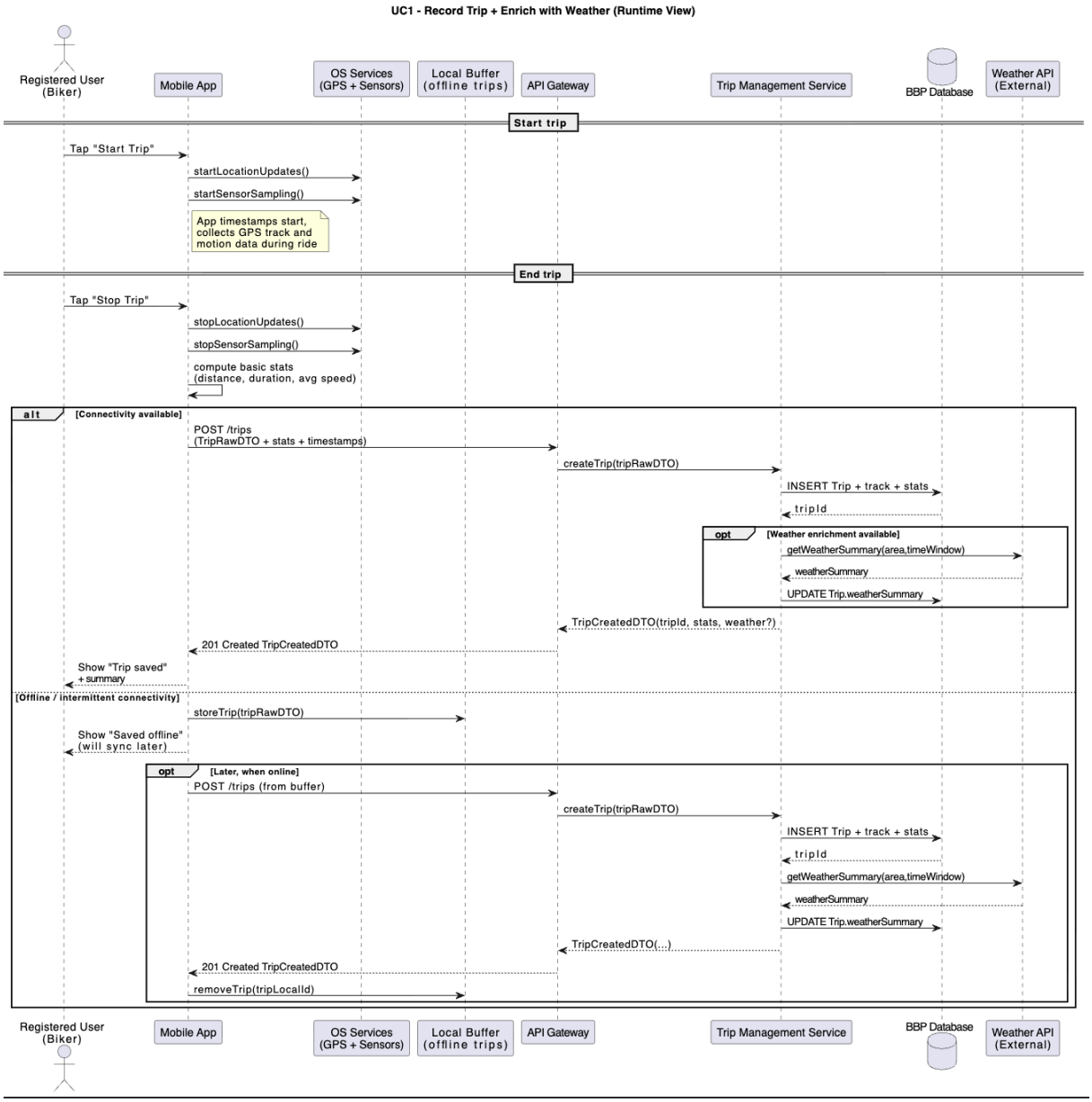


Figure 5: Runtime View of UC1

In this interaction, a registered biker records a trip using the mobile application. The app starts acquiring GPS and motion-related data through OS services. Once the user stops recording, the app computes basic statistics such as duration, distance, and average speed. If connectivity is available, the client uploads the trip data to the backend through the API gateway. The Trip Management Service stores the trip and related statistics in the database and enriches the record with meteorological information retrieved from an external Weather Information Service for the trip time window and area. In case of intermittent connectivity, the client stores the trip in a local buffer and synchronizes it later when the network becomes

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

available.

UC3 Manual Path Contribution

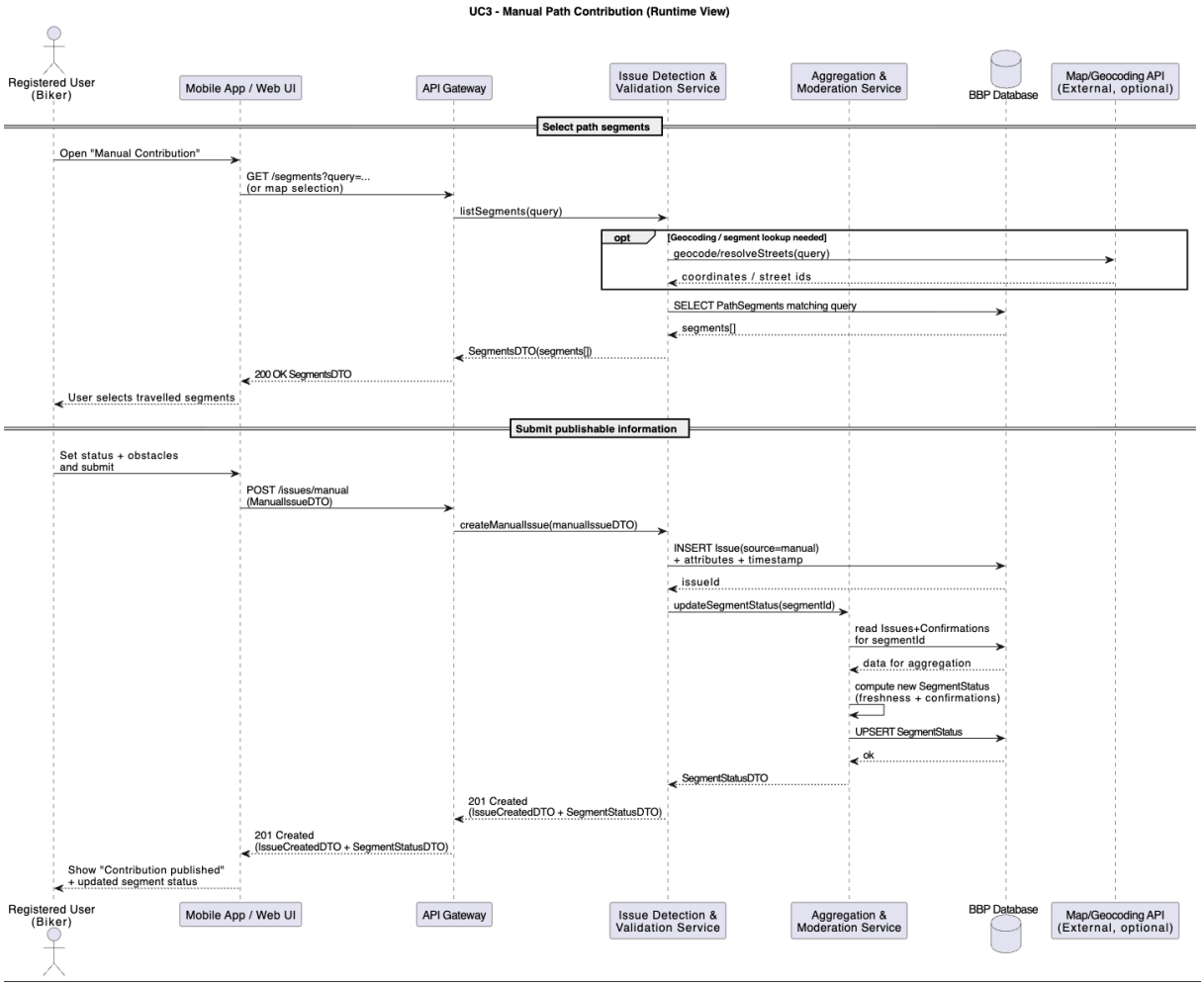


Figure 6: Runtime View of UC3

In this interaction a registered biker manually contributes publishable information about one or more path segments. The client first retrieves candidate segments and shows them for selection. When the user submits the contribution, the backend stores it as a manual Issue. To keep public information consistent and up to date, the system then updates the aggregated publishable representation of the segment by recomputing the SegmentStatus using existing contributions and their recency/confirmation information and stores the updated SegmentStatus in the database. Finally, the system returns confirmation of the publication and the updated segment status to the user.

UC4 Automatic Detection + User Confirmation/Correction

After a trip is recorded, the mobile client uploads a telemetry summary (e.g., travelled segments and derived motion features) to the backend through the API gateway. The Issue Detection & Validation Service stores this data and executes an automatic detection procedure to infer candidate issues, which are

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

stored as Issues in a pending-validation state. When the user opens the validation screen, the client retrieves the pending candidates and displays them. The user can then confirm, reject, or refine a candidate issue. The backend stores the user's decision as a Confirmation and updates the Issue state accordingly. For confirmed/refined issues, the system triggers an update of the publishable aggregated information by recomputing the SegmentStatus for the affected segment and storing the updated SegmentStatus in the database, so that it can be used for future path ranking and visualization.

G5 / F6 Aggregation and Conflict Resolution

This sequence describes how BBP maintains a consistent publishable view of path conditions by aggregating contributions from multiple users. Aggregation can be triggered either event-driven or periodically to refresh recent segments. The Aggregation & Moderation Service retrieves the relevant evidence for each segment from the database and resolves conflicts by combining confirmation count and recency (freshness) into an updated SegmentStatus with an associated confidence value. The resulting SegmentStatus is persisted and becomes the authoritative aggregated information used by other services (e.g., the Path Search Service) to rank and present suggested paths.

2.5 Component interfaces

This section provides a summary of the interfaces exposed by the components of the Best Bike Paths (BBP) system. The internal interface of all the system components is based on the REST-like interface via HTTPS, and the interactions between the system component and external services presuppose the existence of standard APIs that are usually provided by third parties. The names of the methods are abstractly defined with the prefix (`get`, `post`, `put`, `delete`) which mentions the type of operation. The use of overloading is made in cases where the same operation can take different parameters. The names of classes that are used as method parameters are self-explanatory and are in line with the terminology that is presented in the RASD.

Authentication and User Management Interface
This interface controls the user authentication and the session management of registered users.

- `post_register(ClientID, UserData)`
- `post_login(ClientID, Credentials)`
- `post_logout(AuthToken)`

Trip Management Interface
This interface allows the recording, storing, and retrieving of personal trips that have been undertaken by registered users, and submission of telemetry data utilized in auto-detection of issues.

- `post_createTrip(TripRaw)`
- `get_listTrips(User)`
- `get_tripDetails(TripID)`
- `post_submitTelemetry(TripID, TelemetryData)`

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

Manual Contribution Interface
The interface permits registered users to add and update publishable information displaying the status of the bike paths and obstacles.

- `get_segments(Query)`
- `post_createManualIssue(SegmentID, Status, Obstacles, Comment)`
- `put_updateManualIssue(IssueID, UpdatedFields)`
- `delete_manualIssue(IssueID)`

Automatic Issue Validation Interface
The interface is used for accepting, rejecting, or refining automatically raised issues from sensor data collected during trips.

- `get_candidateIssues(TripID)`
- `post_confirmIssue(IssueID, ConfirmationData)`
- `post_rejectIssue(IssueID, Reason)`
- `post_skipIssue(IssueID)`

Path Search and Visualization Interface
The interface allows both registered and unregistered users to search for cycling route options from a given origin to destination and view information on route options.

- `get_searchPaths(Origin, Destination)`
- `get_pathDetails(PathID)`

Aggregation and Moderation Interface (Internal)
This internal interface engages in ensuring a stable publishable version of bike path conditions through convergence of input from several users.

- `post_updateSegmentStatus(SegmentID)`
- `get_segmentStatuses(SegmentIDList)`
- `post_refreshStatuses(TimeWindow)`

External Service Interfaces
The following interfaces are the assumed capabilities of the third-party services utilized by BBP.

Map / Geocoding / Routing Service

- `get_geocode(Location)`
- `get_candidateRoutes(Origin, Destination)`

Weather Information Service

- `get_weatherSummary(Area, TimeWindow)`

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

2.6 Selected architectural styles and patterns

BBP uses a set of simple design ideas that work well with the way it interacts with users. There are two main interaction modes: (i) when a user performs an action and BBP responds immediately, and (ii) when BBP performs work in the background without making the user wait. Consequently, BBP combines synchronous request/response for interactive operations with asynchronous background processing for work that can be safely deferred.

Synchronous interaction style (request/response)

- **Client–Server + RESTful API**: the Mobile App and the Admin Panel invoke backend functionality through a REST-like API over HTTPS/JSON. This covers the majority of interactive use cases (e.g., uploading/reviewing trips, reporting issues, confirming moderation actions, and searching for paths). This design keeps the clients decoupled from the backend implementation details and makes the system easier to evolve and maintain.

- **Logical three-tier architecture (Presentation–Business–Data)**: BBP is organized into three tiers: Presentation (clients), Business (backend logic), and Data (database and cache). This separation keeps the user interface concerns distinct from business rules and from storage details, improving understandability and maintainability.

- **Modular monolith backend**: the Business tier is implemented as a single deployable application with clear internal module boundaries. This supports cohesive behavior (e.g., consistent handling of publishable segment status) while keeping the backend evolvable as the system grows.

- **Layered backend (Controller–Service–Repository)**: internally, the backend follows a layered structure:

  - **Controller/API** handles HTTP requests and access control.
  - **Service/Domain** applies the business rules.
  - **Repository** encapsulates data access to the database and cache.

  This reduces coupling across concerns and keeps persistence details separate from the domain logic.

- **Adapter/Gateway pattern for external integrations**: external providers (e.g., maps and traffic updates) are accessed through an Adapter Layer. This centralizes provider-specific details and failures (timeouts, partial responses, format differences) so the rest of the system can remain simpler and more stable.

- **Caching (client-side and backend)**:

  - **Client-side offline cache**: the Mobile App temporarily stores traces and reports on the device when connectivity is poor, and synchronizes them to the server when the network is available again. This keeps the user experience responsive even under intermittent connectivity.
  - **Backend cache**: the Data tier includes caching for data that is read frequently or is expensive to compute or fetch. The cache is accessed through the Repository so caching does not leak into the business logic.

Asynchronous interaction style (background processing)

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

- **Asynchronous messaging (Queue + Worker)**: BBP uses a queue/worker pattern for work that is slow, bursty, or not required to complete before responding to the user (e.g., enrichment and automatic detection). This preserves responsiveness for interactive operations and decouples request handling from background processing while still persisting outcomes through the same Repository path as the synchronous workflow.

2.7 Other design decisions

Authentication and authorization policy Protected operations require an authenticated user session. Each request to the backend includes an authentication token transmitted securely over HTTPS. Moderator capabilities are exposed through the Admin Panel, and the backend enforces role-based authorization checks to ensure that only users with the appropriate role can execute privileged operations.

Offline synchronization policy The Mobile App applies a *store-and-forward* strategy for trip tracks and reports: when offline, it stores data locally and forwards it once connectivity is restored. Client submissions are designed to be safe to retry (idempotent) using stable identifiers, so temporary connectivity issues do not result in duplicated records on the backend.

SegmentStatus consistency decision For each road segment, the system maintains a single shared *SegmentStatus*. Incoming reports and confirmations update this status according to consistency rules. When the system observes conflicting or non-matching information, it flags the segment for manual review. This prevents the system from presenting different statuses for the same road at the same time and helps keep SegmentStatus accurate and up to date.

Background job execution semantics The Worker executes background tasks such as enrichment, detection, and periodic refresh. Background jobs are designed to be safely retryable and to persist their outcomes through the same Repository path used by synchronous workflows, ensuring results are durable and not lost if a job needs to be re-executed.

External-service resilience policy Calls to external services are performed defensively (timeouts and limited retries when safe). If optional context such as weather or traffic data cannot be retrieved, BBP still returns core results (e.g., route options) and marks the missing enrichment as unavailable rather than failing the entire request.

Caching policy Caching is applied only to read-heavy or expensive computations (e.g., reusable ranking inputs or frequently requested route-related results). Cached entries are refreshed or invalidated when the underlying SegmentStatus changes to reduce the risk of serving stale information.

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

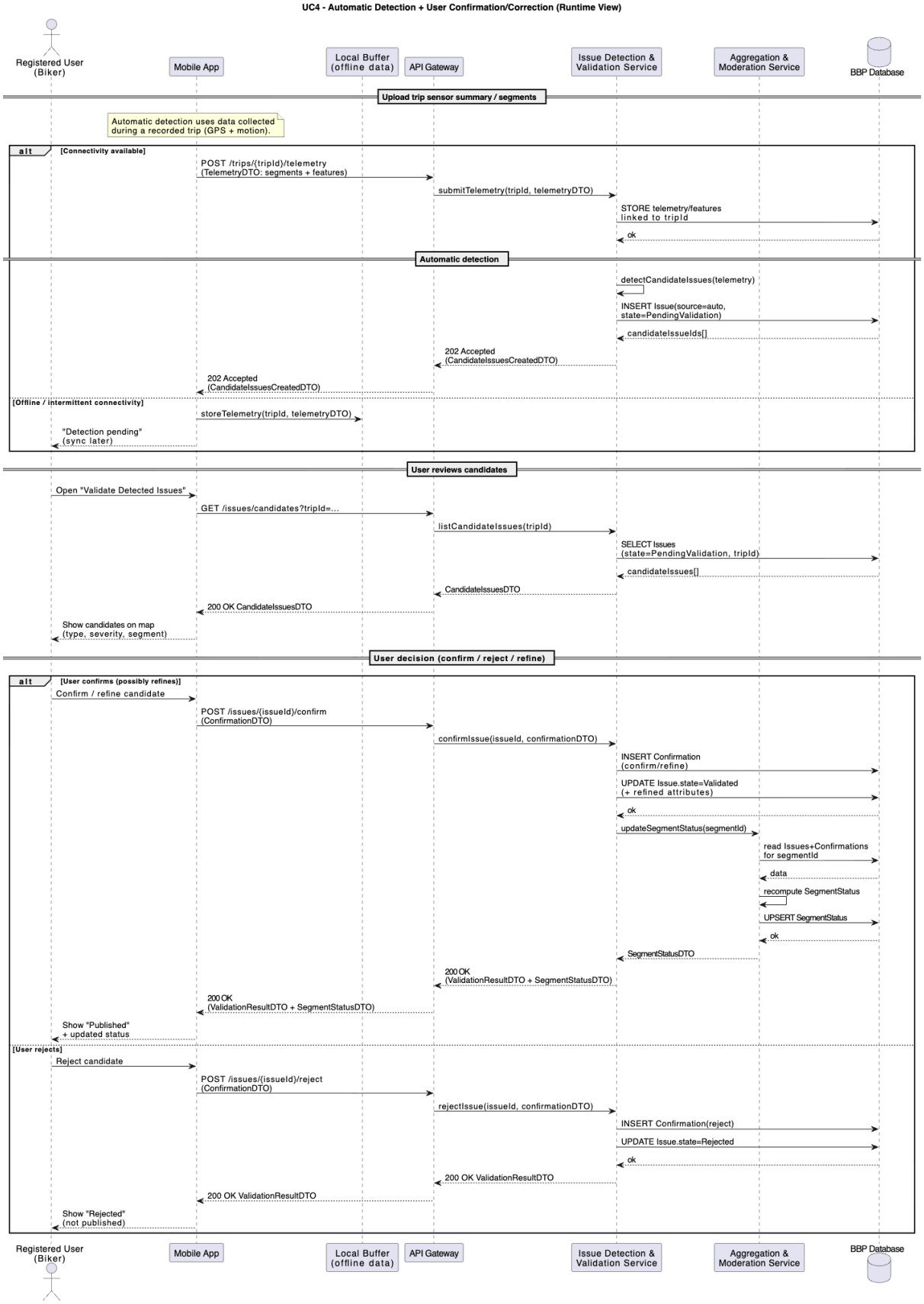


Figure 7: Runtime View of UC4

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

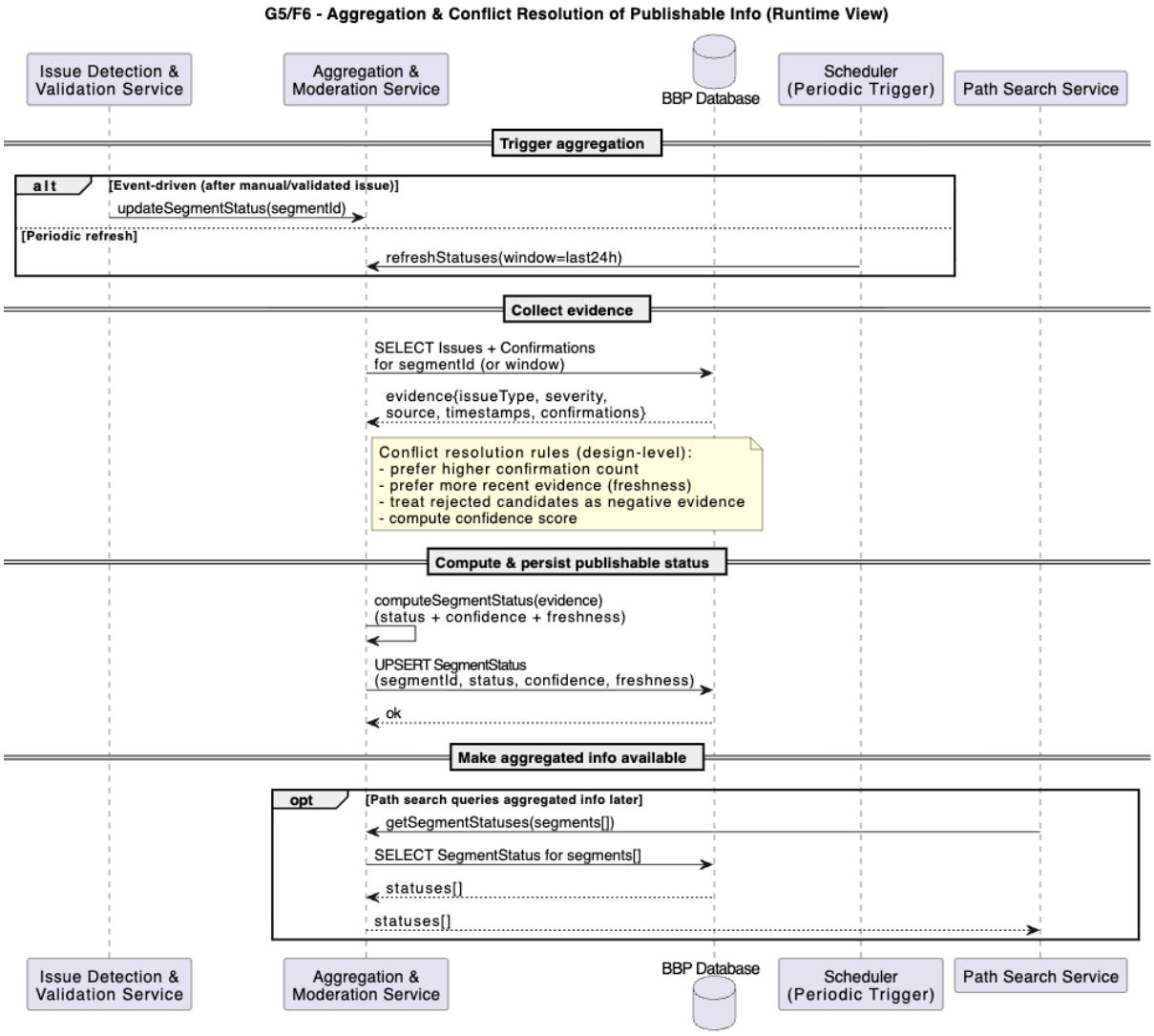


Figure 8: Runtime View of G6/F6

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

3 User Interface Design

This section describes the design of the user interface of the *Best Bike Paths (BBP)* system. The user interface is conceived as a mobile application and is designed to support the interaction scenarios defined in the Requirement Analysis and Specification Document (RASD). Given the context of use of the application, which is mainly during cycling activities, the interface prioritizes clarity, immediate feedback, and minimal user interaction while moving, delegating confirmation and editing tasks to moments when the user is stationary.

3.1 Overview

The BBP user interface is centered around a map-based mobile view, which represents the primary interaction point between the user and the system. Through the map interface, users can visualize bike paths, recorded trips, detected or reported issues, and suggested routes. All core functionalities of the system, such as trip recording, path search, and path information contribution, are accessible through clearly identifiable actions available on the main interface.

3.2 Main User Interfaces

Accordingly, the following are the main user interfaces concerning interaction patterns and use case depicted and designed as regards our RASD.

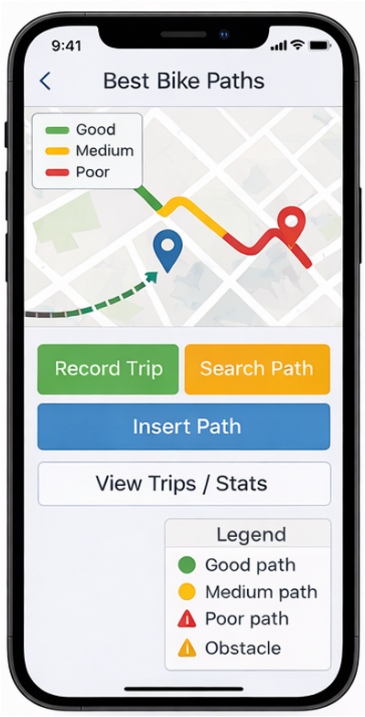


Figure 9: Main map interface of the BBP mobile application

Figure 9 represents the main map interface of the BBP mobile application. This interface, which is the only one accessible to users, serves as an entry point for several use cases that are listed in the RASD, such as recording a new trip, searching paths and inserting manually of path-related information. The map represents bike paths as different color-coded lines and point markers are used to show events like reported issues or obstacles.

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

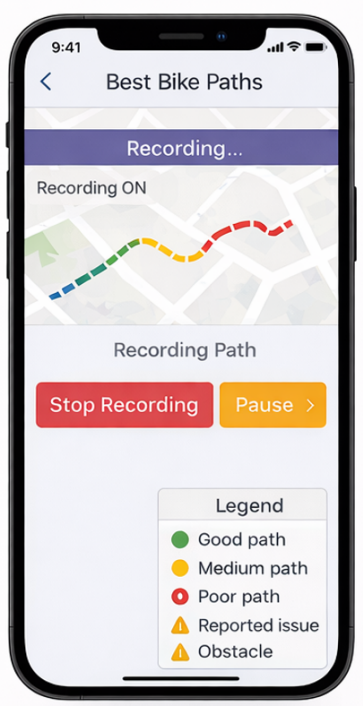


Figure 10: Trip recording interface during a cycling activity

Figure 10 illustrates the trip recording interface, which realizes the *Record Trip* use case described in the RASD. When recording a trip, the UI will show the current path overlaid on the map with easy to see visual cues of whether the app is still recording. User interface is kept to a minimum on purpose - the only thing you need to do will be tap couple buttons every now and then, e.g. when stopping or pausing recording.

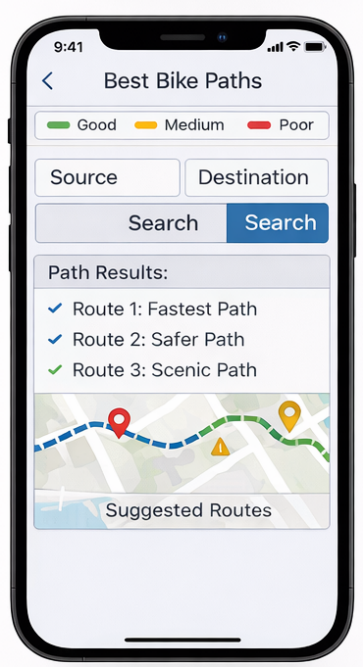


Figure 11: Path search and visualization interface

Figure 11 shows the path search interface, corresponding to the *Search Best Paths* interaction sce-

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

narios defined in the RASD. You can enter a source and destination using this interface, and the system would show one or multiple recommended bike paths on the map. Recommended paths are drawn in different colors based on quality, and additional markers are placed to mark troubled spots and useful information that the paths pass by.

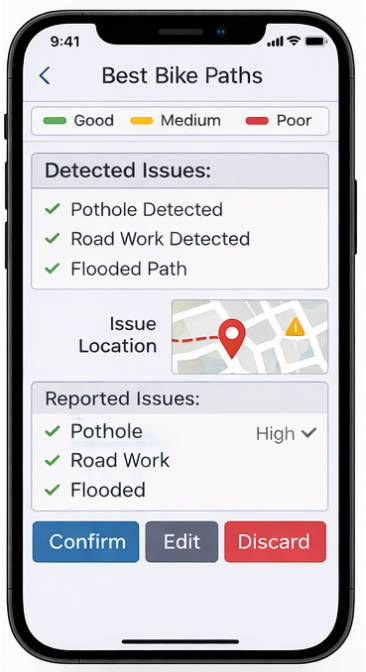


Figure 12: Interface for confirmation of automatically detected issues

Figure 12 represents the interface used for the confirmation of automatically detected issues, as defined in the RASD. After a trip has been made, this interface will allow the users to recognize and explore detected defects on the maps as well as to access further information like issue type and severity. Users can confirm, edit, or reject the identified information before it is published and shared among other users.

3.3 Key Screens and Workflows

The user interaction with the BBP is organized over a set of principal screens which enable the core workflows presented in the RASD. The main map interface shows that each player will start from the main point of cooperation, which is the main gate opened.

The usual method for recording a trip would begin with the user on the primary map page, where they would start to record their cycling. While you take a trip, the recording interface projects your current path while minimizing user interaction. Once the journey has finished, the user is taken to the problem confirmation screen, where automatically recognized problems can be reviewed before confirmation and publishing.

Path search is another important workflow. By interacting with the main map screen, a user can input through the path search interface a start and end point of his or her bike travel and see one or more recommended paths that are ranked based on their fitness to meet the user's needs. This process of manual path information input is treated in the same way as above, with a clear distinction between the main interface and the GUI specific for data insertion and confirmation.

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

3.4 Interface Conventions and Legends

The figures with the graphical user interfaces presented in this section set visual standards for showing information to the user. These conventions are implemented in all the interfaces of the BBP mobile application and their summaries are provided to facilitate the understanding of what is depicted by certain graphics.

Path Color Coding Cycleways on the map are colored differently to indicate their class or function. Green segments are trails marked as 'highly suitable' (based on a combination of user input and confirmed reports). The yellow segments denote moderately suitable routes that pose minor or some restrictions. Red lines indicate weak pathways, which means that there might be significant problems, or unsuitable habitat. The colour coded pattern allows riders to instantly assess the potential quality of available routes.

Point Markers Markers are employed in the map to emphasize certain locations involving events or pertinent conditions therein. The markers can depict automatically detected problems, bike-unfriendly situations reported by a user and other interesting items that might be along the bike path. Each marker is set on the location at that geographic point, and looks different from path segments for users to catch their eye.

Issue and Obstacle Indicators Different marker shapes serve to indicate the different types of issues. For instance, warning symbols may represent obstructions like potholes or road works, and location pins to place the position of detected or reported problems. These indicators help the user learn about critical points on a path and appreciate what type of problem was reported.

Information Panels Information panels show more details or supplementary info about paths, problems or recording trips. Such panels may have features like path name, length of the panel, estimated quality or type and severity of issue. These panels convey structured context that empowers users to make decisions informed by the information, but without cluttering the map view.

Recording Status Indicators As recording is taking place, the user interface provides clear visual signs of which status the current recording is in. Such cues can be indicator lights, status indicators of the recording devices or interface items that indicate the activation of the recording function. This way, there is always feedback to the user if a cycling activity is being recorded.

Legend Box There is also a legend box on the interface; it summarizes the significance of colors, markers, and symbols used in the map. This legend serves as a shorthand explanation that enables viewers to interpret the visual language of the application at-a-glance, without any extra instruction.

3.5 Accessibility and Interaction Notes

The BBP interface is especially mindful on the accessibility paradigm and takes into consideration mobile interaction limitations. As the app is primarily used while riding a bike, the design reduces interactions to be as simple as possible through and in motion experience using big touch areas and immediate visual feedback. Colour legend offers information on the path quality, but both coloured-text labels and typical legends values lead into that all information are meaningful in every color perception context. Further, by processing confirmation and editing operations during times when the user has stopped moving, cognitive load is lowered, and safety is enhanced. Such design considerations are intended to help keep the application usable in real-world usage scenarios.

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

3.6 Navigation Flow

The navigation among the different user interfaces is designed to be simple and coherent. For the primary user interface, users can move to the trip recording screen to begin a cycling activity or path searching screen for route calculation. After a tour is finished, users are directed to the issue confirmation interface to confirm auto-extracted information. Supplementary features (e.g., input path manually and trip statistics) are available as clickable actions in the main interface. This navigation framework provides easy access to the main functions of the BBP system, while maintaining a clear separation among the data collection, validation and display processes.

The user interface figures presented in this section are conceptual mockups and they were created to illustrate the structure and interaction of the mobile interfaces and produced using a UI prototyping tool [1].

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

4 Requirements Traceability

This section relates the functional requirements defined in the RASD (IDs `RE-F*`) to the main DD artifacts that realize them: (i) architectural components and their responsibilities, (ii) component interfaces, (iii) runtime interaction diagrams, and (iv) user interface elements. The purpose is to demonstrate that each requirement is supported by concrete design choices and a clear end-to-end execution path.

RASD Req. ID	Requirement intent (from RASD)	DD realization (components, interfaces, runtime, UI)
F2 - Trip Recording and Review		
RE-F2-1	The system shall allow users to record a biking trip.	**Components:** Mobile App, Trip Management, Backend API, Database. **Runtime:** Trip recording and upload flow. **UI:** Trip recording screen.
RE-F2-2	The system shall store recorded trips for later access.	**Components:** Trip Management, Database. **Runtime:** Persistent storage after trip completion. **UI:** Trip history and summary screens.
RE-F2-3	The system shall acquire GPS data during a biking session.	**Components:** Mobile App, GPS sensor services. **Runtime:** Continuous GPS acquisition during trip recording. **UI:** Live position feedback during recording.
RE-F2-4	The system shall acquire motion sensor data during a biking session.	**Components:** Mobile App, Motion sensors. **Runtime:** Motion data sampling during recording. **UI:** Background acquisition indicators.
RE-F2-5	The system shall associate recorded trips with the corresponding user.	**Components:** Authentication service, Trip Management. **Runtime:** User identity attached to trip metadata. **UI:** User-specific trip lists.
RE-F2-7	The system shall enrich recorded trips with contextual information when available.	**Components:** Trip Management, Weather Information Service. **Runtime:** Contextual enrichment after trip upload. **UI:** Contextual data displayed in trip details.
RE-F2-8	The system shall allow users to review previously recorded trips.	**Components:** Trip Management, Backend API, Database. **Runtime:** Trip retrieval flow. **UI:** Trip history and detail views.
F3 - Manual Contribution		
RE-F3-1	The system shall allow users to manually contribute information about path segments.	**Components:** Path Contribution logic, Backend API, Database. **Runtime:** Manual contribution submission flow. **UI:** Path contribution screen.

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

RASD Req. ID	Requirement intent (from RASD)	DD realization (components, interfaces, runtime, UI)
RE-F3-2	The system shall allow users to specify the condition or status of path segments.	**Components:** Path Contribution logic, Aggregation service. **Runtime:** Validation and storage of segment status. **UI:** Segment status selection controls.
RE-F3-3	The system shall allow users to publish contributed path information.	**Components:** Aggregation and Moderation service. **Runtime:** Publication after validation. **UI:** Submit and confirmation actions.
RE-F3-4	The system shall allow users to modify or remove previously submitted contributions.	**Components:** Path Contribution logic, Database. **Runtime:** Update and deletion flows. **UI:** Contribution management interface.
F4 - Issue Detection and Validation		
RE-F4-1	The system shall automatically detect potential issues along recorded trips.	**Components:** Issue Detection service, Mobile App. **Runtime:** Automatic detection after data upload. **UI:** Issue notifications.
RE-F4-2	The system shall present detected issues to users for validation.	**Components:** Issue Validation service, Backend API. **Runtime:** Retrieval of candidate issues. **UI:** Issue validation screen.
RE-F4-3	The system shall allow users to confirm detected issues.	**Components:** Issue Validation service, Aggregation service. **Runtime:** Confirmation flow updating issue state. **UI:** Confirm controls.
RE-F4-4	The system shall allow users to reject detected issues.	**Components:** Issue Validation service. **Runtime:** Rejection and storage of decision. **UI:** Reject controls.
RE-F4-5	The system shall ensure that only validated issues are exposed to other users.	**Components:** Aggregation service, Path Search service. **Runtime:** Filtering of non-validated issues. **UI:** Map markers for validated issues only.
F5 - Path Search and Visualization		
RE-F5-1	The system shall allow users to search for biking paths between two locations.	**Components:** Path Search service, Routing services. **Runtime:** Path search execution. **UI:** Search controls on the main map.
RE-F5-2	The system shall compute feasible biking paths between origin and destination.	**Components:** Path Search service, Aggregation service.

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

RASD Req. ID	Requirement intent (from RASD)	DD realization (components, interfaces, runtime, UI)
		Runtime: Routing and path computation flow. **UI:** Map-based path visualization.
RE-F5-3	The system shall rank computed paths according to quality indicators.	**Components:** Aggregation service, Path Search service. **Runtime:** Path scoring and ranking. **UI:** Ranked list of paths.
RE-F5-4	The system shall allow users to visualize suggested biking paths.	**Components:** Client UI, Path Search service. **Runtime:** Path retrieval and rendering. **UI:** Main map view with overlays.
RE-F5-5	The system shall allow users to view detailed information about suggested paths.	**Components:** Path Search service, Database. **Runtime:** Retrieval of path details. **UI:** Path detail screen with legend.

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

5 Implementation, Integration and Testing

In this section, the application of the system for implementation, integration and test is described in detail. The goal of this plan is to eliminate integration risks and to have the final system do what it is supposed to after one has put together all subsystems. The activities reported in this category are planned activities and shall be performed in the development and validation process.

5.1 Implementation Strategy

The system will be constructed as an ensemble of loosely coupled components. Each module will represent a certain role in the architecture. Components will communicate through well-defined interfaces. This modular approach will support parallel development and simplify testing. External services, like routing or weather providers will be accessed through specialized interfaces, which will reduce coupling and serve to improve testability.

5.2 Feature Identification

This section lists the key features of the system which will drive integration and testing work. When the complete behavior is correct, a feature will be marked as done and then back through the backend to user interface.

The identified features will include:

- user management, including registration and authentication;
- trip recording and review;
- manual contribution of path information;
- issue detection and validation;
- path search and visualization;
- aggregation and moderation of community data.

These will be used to characterize the representative integration threads and system-level test scenarios.

5.3 Component Testing

Component testing will verify the behavior of individual components in isolation, before any form of system-level integration is performed.

Visual Conventions. The following visual conventions are used in this section. Software components or subsytems are shown as rectangular boxes. Solid-line boxes represent components which are actually implemented and running. Dashed-line boxes illustrate stubs (simulated components are employed during testing to replace external dependencies. Control or Data flow are represented with arrows between components and indicate the direction of invocation.

Colors are only used to make the text more readable and do not carry any further semantics or architectural constraints.

During component testing, all external dependencies of the component under test will be replaced by stubs. The number of stubs is not fixed and depends on the number of external dependencies: one stub will be introduced for each external system that is not part of the component under test.

Figure 13 illustrates the planned component testing setup. The figure shows a test driver invoking the component under test, while all external dependencies are simulated using stubs. This setup will be

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

used in testing the internal logic of the component without depending on outside systems like databases or third-party services.

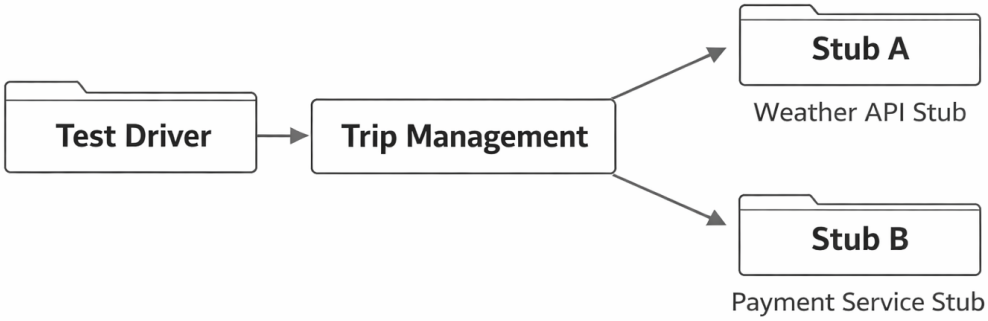


Figure 13: Component testing using a test driver and stubs.

The caller of the component under test will be simulated by a test driver and will trigger its operations. Stubs will return predefined responses and will make tests deterministic and repeatable.

5.4 Incremental Integration Strategy

After component testing, system integration will be performed incrementally. Subsystems will not be integrated all at once.

Figure 14 shows the planned incremental integration strategy. Integration will proceed through successive steps, where the number of real subsystems will gradually increase. In early steps, only core infrastructure will be real, while other subsystems and external services will be replaced by stubs. As integration progresses, stubs will be progressively replaced by real components.

This approach will make it possible to identify integration problems early and to limit the impact of faults during later development stages.

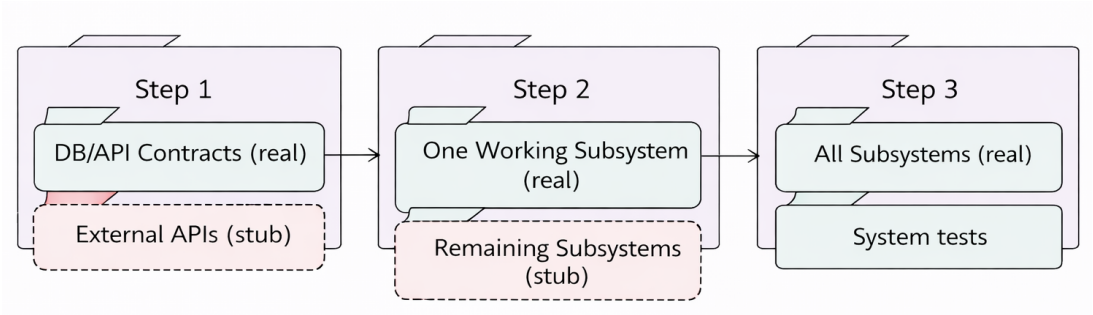


Figure 14: Incremental integration strategy with progressive replacement of stubs.

5.5 Thread-Based Integration

Apart from this incremental integration, a thread-based integrated approach will be adopted.

A thread is a coherent visible to the user operation that ranges many components. Thread-based integration will focus on validating full execution paths rather than individual subsystems.

Figure 15 illustrates the integration of a representative thread, namely the Path Search and Visualization feature. The diagram illustrates how the client interface, backend services, stateful storage and any external services collaborate to deliver a single functionality.

This thread will be tested to allow verification of interactions across all involved components and should identify the issues which cannot be detected via component testing alone.

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

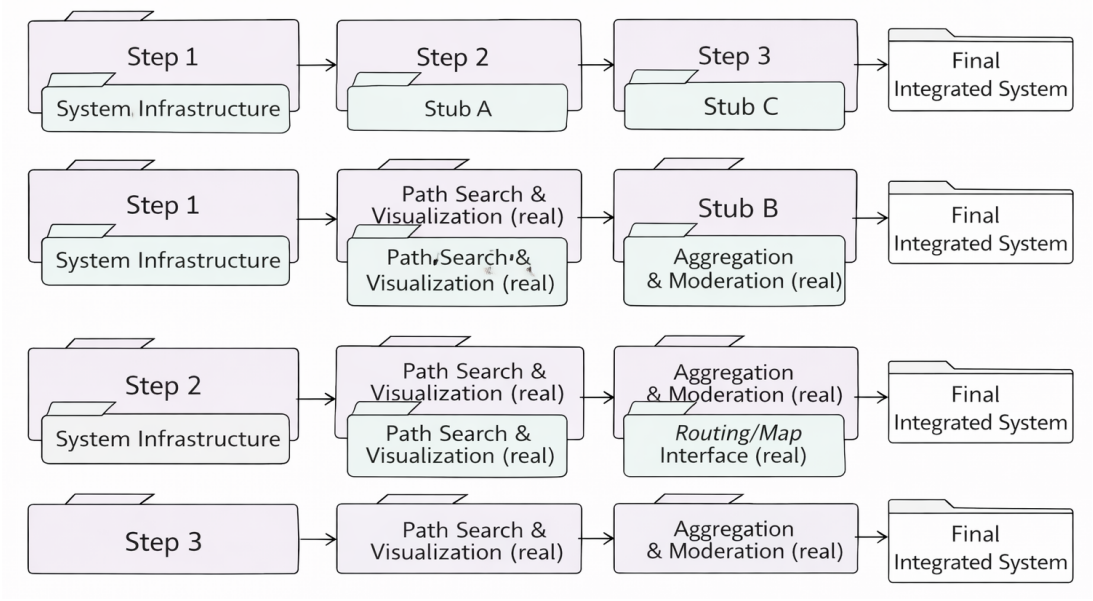


Figure 15: Thread-based integration of the Path Search and Visualization feature.

5.6 Final System Integration

Once all subsystems and representative threads are integrated, the system will reach its final configuration.

Figure 16 presents the fully integrated system. At this stage, all components will be real and no drivers or stubs will be used. The figure represents the system as it will be executed in production.

System testing will be performed on this configuration in order to validate overall system behavior and interactions from the user's perspective.

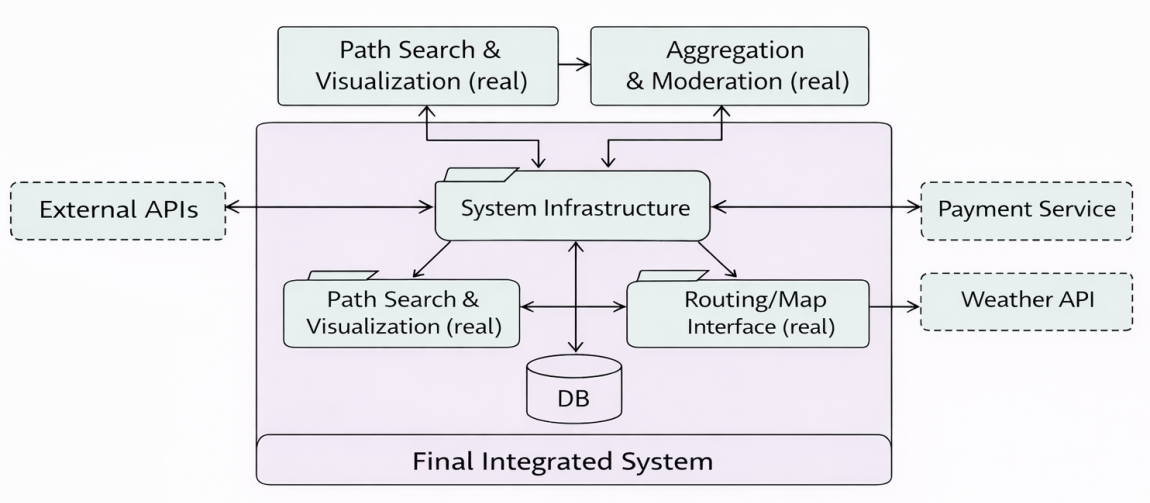


Figure 16: Final system configuration after full integration.

5.7 Testing Summary

Testing will be conducted in various levels. Individual modules will be validated by component testing in isolation. Integration testing will validate interactions among subsystems using incremental and thread-based strategies. System tests will verify the behavior of the fully integrated system. This structured testing plan would enhance software quality and minimize integration risks.

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

6 Effort Spent

6.1 Shuvro Ahmed

Task	Hours
Design Document structure setup	1
Architectural overview review and alignment with RASD	2
Deployment View - design and refinement	5
Deployment View description and node explanations	2
Runtime View - UC5/UC6 Path Search	4
Runtime View - UC1 Record Trip + Weather Enrichment	4
Runtime View - UC3 Manual Path Contribution	3
Runtime View - UC4 Automatic Detection + User Confirmation	5
Runtime View - Aggregation and Conflict Resolution (G5/F6)	3
Component Interfaces - interface identification and consolidation	3
Component Interfaces - final bullet-style interface specification	2
Consistency checks between RASD and DD (goals, use cases, services)	2
Fix LaTeX issues, figures, and repository organization	3

6.2 Umme Humaira Himi

Task	Hours
Introduction section drafting	2
Design Document review and alignment with RASD	2
User Interface Design	4
UI legends, conventions, and accessibility notes	2
UI mockups and screen refinement	4.5
Requirements Traceability analysis	5
Traceability mapping and table preparation	3
Implementation and integration planning	3.5
Integration and testing figures preparation	3
Cross-section consistency checks (Sections 3, 4, 5)	1.5
Final review and LaTeX formatting	1

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

6.3 Ahmed Hassan

Task	Hours
Project Initialization fixes	1
Architectural overview	12
Component View	16
Selected Architecture Style and Pattern	3
Other Design Choices	3

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved

7 References

References

[1] Figma: Collaborative interface design tool. https://www.figma.com. Accessed: January 2026.

[2] JGraph. diagrams.net, draw.io, 10 2021.

Copyright © 2025, Umme Humaira Himi, Ahmed Hassan, Shuvro Ahmed – All rights reserved